

2.5 硬件描述语言 Verilog HDL 基础

2.5.1 VerilogHDL的基本结构

2.5.2 逻辑功能的仿真与测试

2.5.3 基本语法规则

2.5.4 数据类型

2.5.5 运算符及其优先级

2.5.6 Verilog内部的基本门级元件

2.5 硬件描述语言 Verilog HDL 基础

硬件描述语言 HDL(Hardware Description Language)
类似于高级程序设计语言。它是一种以文本形式来描述数字系统硬件的结构和行为的语言，用它可以表示逻辑电路图、逻辑表达式，复杂数字逻辑系统完成的逻辑功能。 HDL 是高层次自动化设计的起点和基础。

计算机对 HDL 的处理：

逻辑仿真 是指用计算机仿真软件对数字逻辑电路的结构和行为进行预测。仿真器对 HDL 描述进行解释，以文本形式或时序波形图形式给出电路的输出。在仿真期间如发现设计中存在错误，可以对 HDL 描述进行及时的修改。

逻辑综合 是指从 HDL 描述的数字逻辑电路模型中导出电路基本元件列表以及元件之间的连接关系（常称为门级网表）的过程。类似对高级程序语言设计进行编译产生目标代码的过程。产生门级元件及其连接关系的数据库，根据这个数据库可以制作出集成电路或印刷电路板 PCB。

2.5.1 Verilog 程序的基本结构

模块是 Verilog 描述电路的基本单元。对数字电路建模时，用一个或多个模块。不同模块之间通过端口进行连接。

- 1、每个模块以关键词 module 开始，以 endmodule 结束。**
- 2、每个模块先要进行端口的定义，并说明输入（input）和输出（output），然后对模块功能进行描述。**
- 3、除了 endmodule 语句外，每个语句后必须有分号。**
- 4、可以用 /* --- */ 和 //..... 对程序的任何部分做注释。**
- 5、逻辑功能的描述方式有三种不同风格：结构描述方式（门级描述方式）数据流描述方式，行为描述方式。**

模块定义的一般语法结构如下：

module 模块名 (端口名1, 端口名2, 端口名3, ...);

端口类型说明(input, outout, inout);

参数定义(可选);

数据类型定义(wire, reg等);

说明部分

实例化低层模块和基本门级元件;

连续赋值语句(assign);

过程块结构(initial和always)

行为描述语句;

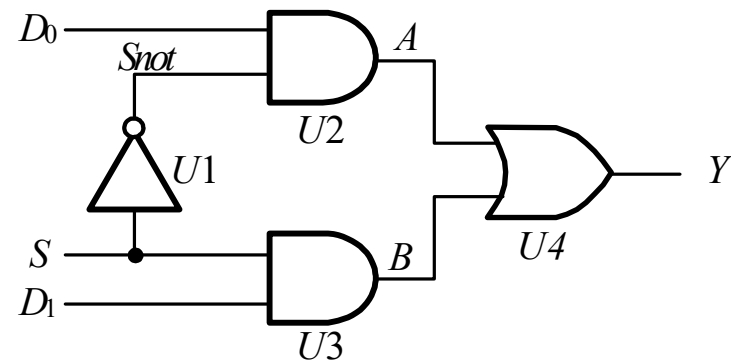
逻辑功能描述部分, 其顺序是任意的

endmodule

例 用门级描述建立模型

模块名

```
module mux2to1(D0, D1, S, Y);  
  input D0, D1, S; // 定义输入信号  
  output Y; // 定义输出信号  
  wire Snot, A, B; // 定义内部节点信号数据类型  
  // 下面对电路的逻辑功能进行描述  
  not U1(Snot, S);  
  and U2(A, D0, Snot);  
  and U3(B, D1, S);  
  or U4(Y, A, B);  
endmodule
```



端口类型说明

数据类型说明

电路结构描述

例 用数据流描述方式建立模型

$$Y = D_0 \cdot \bar{S} + D_1 \cdot S$$

```
module mux2to1_df(  
  input D0, D1, S ,  
  output wire Y  
);
```

修订版本 Verilog-2001/2005 标准的写法

```
// 电路功能描述
```

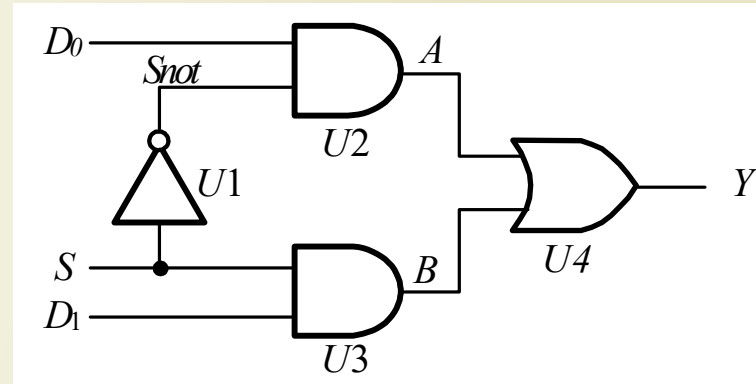
数据类型说明

```
assign Y = (~S & D0) | (S & D1); // 表达式左边 Y 必须是 wire
```

型

```
endmodule
```

电路结构描述

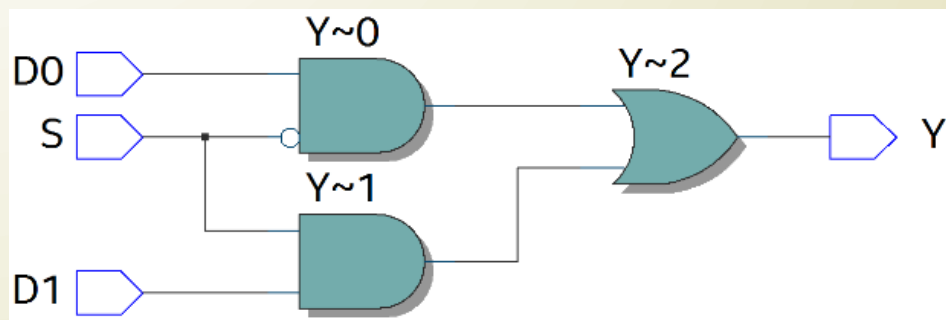


注意，在 assign 语句中，左边变量的数据类型必须是 wire 型。

逻辑综合

将数据流描述的代码送到逻辑综合软件进行综合，得到如图 2.5.5 所示的逻辑图

这是 IntelFPGA 开发软件 Quartus Prime 17.1 进行逻辑综合的结果。



数据流描述代码逻辑综合的结果

2.5.2 逻辑功能的仿真与测试

逻辑电路的设计块完成后，就要测试这个设计块描述的逻辑功能是否正确。为此必须在输入端口加入测试信号，而从其输出端口检测其结果是否正确，这一过程常称为搭建测试平台。根据仿真软件的不同，搭建测试平台的方法也不同。

ModelSim 软件为例，首先建立激励块
激励块大致由三部分组成：

- 一、实例引用被测试的模块（即设计块）。
- 二、给输入信号赋值，即激励信号。
- 三、指定测试结果的显示格式，并指定输出文件名。

2.5.2 逻辑功能的仿真与测试

激励块代码

```
// 文件名: test_mux2to1_df.v
`timescale 1ns/1ns           // 时间单位为 1 ns , 精确度为 1ns
module test_mux2to1_df;      // 激励块, 没有端口列表
    reg PD0, PD1, PS;        // 声明输入信号
    wire PY;                  // 声明输出信号

    // 实例引用设计块
    mux2to1_df t_mux (PD0, PD1, PS, PY); // 按照端口位置连接
    initial begin              // 激励信号
        PS = 0; PD1 = 0; PD0 = 0;    // 语句 1
        #5 PS = 0; PD1 = 0; PD0 = 1; // 语句 2 ( #5 代表延迟 5 ns )
        #5 PS = 0; PD1 = 1; PD0 = 0; // 语句 3
        ....( 这里省略了语句 4-8 , 写完整 )
        #5 PS = 0; PD1 = 0; PD0 = 0; // 语句 9
        #5 $stop;                     // 语句 10
    endmodule
```

2.5.2 逻辑功能的仿真与测试

激励块代码（续）

```
initial begin // 输出部分
```

```
    $monitor($time, ":\tS = %b\tD1 = %b\tD0 = %b\tY =  
    %b",PS,PD1,PD0,PY);
```

```
    end
```

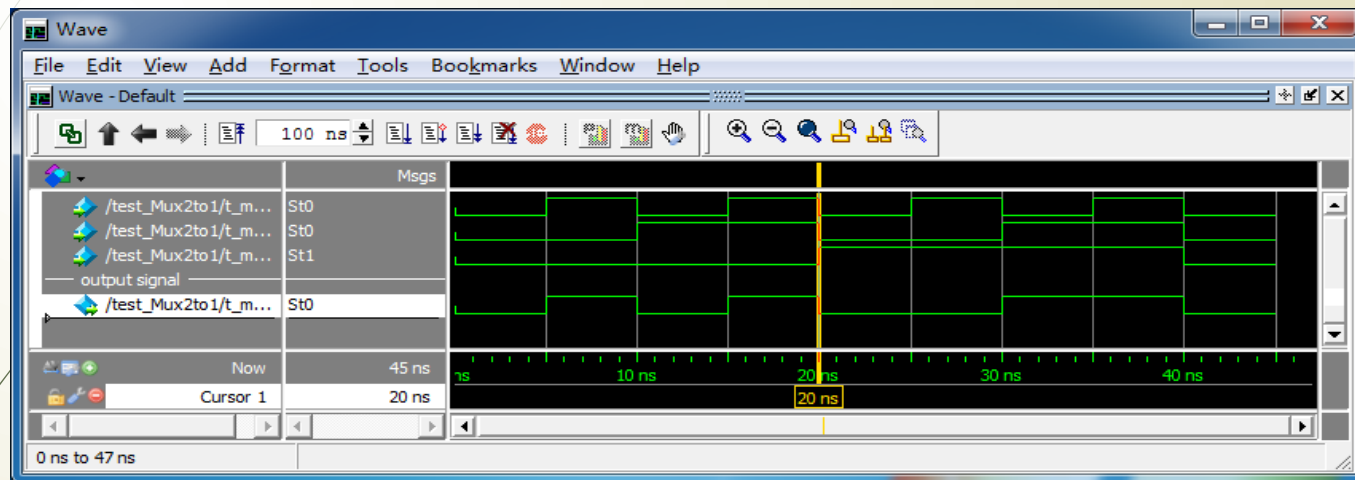
```
endmodule
```

其中，

- `$monitor`、`$time` 和 `$stop` 为系统任务
- `$monitor` 将信息以指定的格式输出到屏幕上，双引号内是要显示的内容，
- `%b` 代表它后面的信号用二进制格式显示，
- `\t` 代表水平制表符，
- `$time` 将返回当前的仿真时间，
- `$stop` 为停止仿真，但不退出仿真环境。附录 A 中将介绍。

2.5.2 逻辑功能的仿真与测试

建立工程项目，编译、仿真，输出波形或文本结果



The Transcript window displays the simulation results. The command `run -all` was executed. The resulting simulation data shows the state of signals S, D1, D0, and Y at various time intervals from 0 ns to 40 ns. A note indicates the simulation stopped at 45 ns due to a break in the module `test_Mux2to1` at line 20.

```
VSIM 3> run -all
#           0: S = 0 D1 = 0 D0 = 0 Y = 0
#           5: S = 0 D1 = 0 D0 = 1 Y = 1
#          10: S = 0 D1 = 1 D0 = 0 Y = 0
#          15: S = 0 D1 = 1 D0 = 1 Y = 1
#          20: S = 1 D1 = 0 D0 = 0 Y = 0
#          25: S = 1 D1 = 0 D0 = 1 Y = 0
#          30: S = 1 D1 = 1 D0 = 0 Y = 1
#          35: S = 1 D1 = 1 D0 = 1 Y = 1
#          40: S = 0 D1 = 0 D0 = 0 Y = 0
# ** Note: $stop      : D:/myIntelFPGA17/task_2.p71/sim/t_Mux2to1_df.v(20)
#   Time: 45 ns   Iteration: 0   Instance: /test_Mux2to1
# Break in Module test_Mux2to1 at D:/myIntelFPGA17/task_2.p71/sim/t_Mux2to1_df.v line 20
VSIM 4>
```

2.5.3 Verilog HDL 的基本语法规则

为对数字电路进行描述（常称为建模），Verilog 语言规定了一套完整的语法结构。

1. 间隔符：Verilog 的间隔符主要起分隔文本的作用，可以使文本错落有致，便于阅读与修改。

间隔符包括空格符（\b）、TAB 键（\t）、换行符（\n）及换页符。

2. 注释符：注释是为了改善程序可读性，在编译时不起作用。

多行注释符（用于写多行注释）：/* --- */；

单行注释符：以 // 开始到行尾结束为注释文字。

3. 标识符和关键词

标识符：给对象（如模块名、电路的输入与输出端口、变量等）取名所用的字符串。以英文字母或下划线开始

如，clk、counter8、_net、bus_A。

关键词：是 Verilog 语言本身规定的特殊字符串，用来定义语言的结构。例如，module、endmodule、input、output、wire、reg、and 等都是关键词。关键词都是小写，关键词不能作为标识符使用。

4. 逻辑值集

为了表示数字逻辑电路的逻辑状态，Verilog 语言规定了 4 种基本的逻辑值。

0	逻辑 0、逻辑假
1	逻辑 1、逻辑真
x 或 X	不确定的值（未知状态）
z 或 Z	高阻态

5. 常量及其表示

常量

整数型

十进制数的形式的表示方法：表示有符号常量

二进制数的形式的表示方法：表示无符号常量

格式为：<+ / - ><位宽><基数符号><数

例如：3'b1x0x、5'o37、8'he3、8'b1001_0011

实数型常量

十进制记数法 如：0.1、2.0、

5.67

科学记数法 如：23_5.1e2、5E -

4

Verilog 允许用参数定义语句，定义一个标识符来代表一个常量，称为符号常量。定义的格式为：

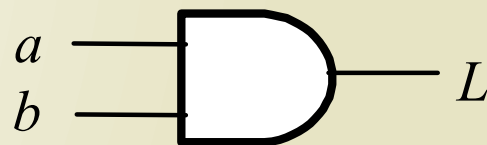
parameter 参数名 1 = 常量表达式 1, 参数名 2 = 常量表达式 2, ...; 如 parameter BIT=1, BYTE=8, PI=3.14;

6. 字符串：字符串是双撇号内的字符序列

2.5.4 数据类型

1、线网类型：是指输出始终根据输入的变化而更新其值的变量，它一般指的是硬件电路中的各种物理连接。

例：网络型变量 L 的值由与门的驱动信号 a 和 b 所决定，即 $L = a \& b$ 。 a 、 b 的值发生变化，线网 L 的值会立即跟着变化。



常用的网络类型由关键词 `wire` 定

※ `wire` 型变量的定义格式如下：

`wire [n-1:0] 变量名 1, 变量名 2, ..., 变量名`

`n;`
例：`wire L;` // 将上述电路的输出信号 L 声明为网络型变

量
量

`wire [7:0] data bus;` // 声明 8-bit 宽的网络型总线变

2、变量类型（之前称寄存器型）

它对应的是具有状态保持作用的电路元件，如触发器寄存器。

寄存器型变量只能在 initial 或 always 内部被赋值。

4 种变量类

抽象描述，
不对应具
体硬件

变量类型	功能说明
reg	用于行为描述，表示存储逻辑值变量
integer	32 位带符号的整数型变量
real	64 位带符号的实数型变量，
time	64 位无符号的时间变量

例： reg clock ; // 定义一个 1 位寄存器变量
reg [3:0] counter; // 定义一个 4 位寄存器变量

2.5.5 运算符及其优先级

1. 运算符

运算符分为算术运算符、逻辑运算符、关系运算符、移位运算符等

类型	符号	功能说明	类型	符号	功能说明
算术运算符	$+$ $-$ $-$ $*$ $/$ $\%$ $**$	二进制加 二进制减 2 的补码 二进制乘 二进制除 取余 指数幂	关系运算符 (双目运算符)	$>$ $<$ $>=$ $<=$ $==$ $!=$ $===$ $!==$	大于 小于 大于或等于 小于或等于 相等 不相等 Case 中的相等 Case 中的不等
位运算符 (双目运算符)	$\sim$ $\&$ $ $ $\wedge$ $\sim$ 或 $\sim\wedge$	按位取反 按位与 按位或 按位异或 按位同或	缩位运算符 (单目运算符)	$\&$ $\sim\&$ $ $ $\sim $ $\wedge$ $\sim$ 或 $\sim\wedge$	缩位与 缩位与非 缩位或 缩位或非 缩位异或 缩位同或
逻辑运算符 (双目运算符)	$!$ $\&\&$ $\parallel$	逻辑非 逻辑与 逻辑或	移位运算符 (双目运算符)	$>>$ $<<$ $>>>$ $<<<$	逻辑右移 逻辑左移 算术右移 算术左移
位拼接运算符	$\{\}$ $\{\{\}$	将多个操作数 拼接成为一个 操作数	条件运算符 (三目运算符)	$?:$	根据条件表达式是否成立, 选择表达式

位拼接运算符

作用是将两个或多个信号的某些位拼接起来成为一个新的操作数，进行运算操作。

设 $A=1'b1$, $B=2'b10$,
则 $\{B,C\} = 4'b1000$
 $\{A,B[1],C[0]\} = 3'b110$
 $\{A,B,C,3'b101\}$
 $=8'b11000101$ 。

对同一个操作数的重复拼接还可以双重大括号构成的运算符 $\{\{\}\}$ 。例如， $\{4\{A\}\}=4'b1111$, $\{2\{A\},2\{B\},C\}=8'b11101000$ 。

位运算符与缩位运算的比较

A : 4'b1010 、

B : 4'b1111 ,

位运算	$\sim A = 0101$ $\sim B = 0000$	$A \& B = 1010$	$A B = 1111$	$A \wedge B = 0101$	$A \sim \wedge B = 1010$
缩位运算	$\& A = 1 \& 0 \& 1 \& 0 = 0$	$\sim \& A = 1$ $\& B = 1$	$ A = 1$ $\sim B = 0$	$\wedge A = 0$ $\wedge B = 0$	$\sim \wedge A = 1$ $\sim \wedge B = 1$

2. 运算符的优先级

优先级的顺序从下向上依次增加。

类型	符号	优先级别	
取反	! ~ -(求2的补码)	最高优先级	
算术	* / + -		
移位	>> << >>> <<<		
关系	< <= > >=		
等于	== != === !==		
缩位	& ~& ^ ^~ ~		
逻辑	&& 		
条件	?:	最低优先级	

条件运算符

是三目运算符，运算时根据条件表达式的值选择表达式。

一般用法：

`condition_expr?expr1:expr2;`

首先计算第一个操作数 `condition_expr` 的值：

- 结果为逻辑 1，则选择第二个操作数 `expr1` 的值作为结果返回。
- 结果为逻辑 0，选择第三个操作数 `expr2` 的值作为结果返回。

2.5.6 Verilog 内部的基本门级元件

门级建模：将逻辑电路图用 HDL 规定的文本语言表示出来。

基本门级元件模型

三态门

多输出门

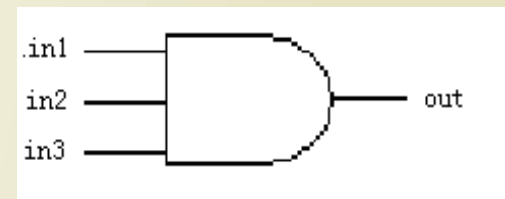
多输入门

元件符号	功能说明	元件符号	功能说明
and	多输入端的与门	nand	多输入端的与非门
or	多输入端的或门	nor	多输入端的或非门
xor	多输入端的异或门	xnor	多输入端的异或非门
buf	多输出端的缓冲器	not	多输出端的反相器
bufif1	控制信号高电平有效的三态缓冲器	notif1	控制信号高电平有效的三态反相器
bufif0	控制信号低电平有效的三态缓冲器	notif0	控制信号低电平有效的三态反相器

1、多输入门

只允许有一个输出，但可以有多多个输入。

调用名
and A1 (out , in1 , in2 ,
in3) ;



and 真值表

x- 不确定状态 z- 高阻态

如果有一个输入为 x 或者 z , 则输出为 x 。

and		输入 1			
		0	1	x	z
输入 2	0	0	0	0	0
	1	0	1	x	x
	x	0	x	x	x
	z	0	x	x	x

or 真值表

or		输入 1			
		0	1	x	z
输入 2	0	0	1	x	x
	1	1	1	1	1
	x	x	1	X	X
	z	x	1	X	X

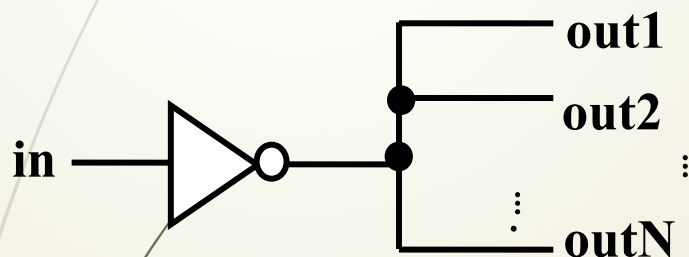
xor 真值表

xor		输入 1			
		0	1	x	z
输入 2	0	0	1	x	x
	1	1	0	x	x
	x	x	x	x	x
	z	x	x	x	x

2、多输出 门

允许有多个输出，但只有一个输入。

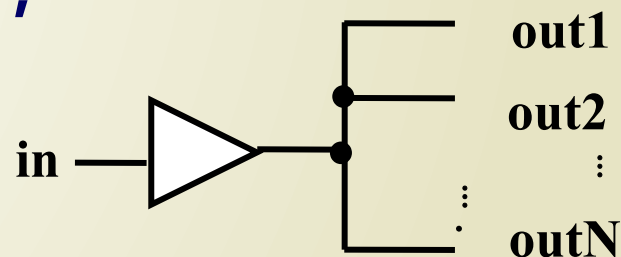
not N1 (out1 , out2 , ...,
in) ;



not 真值表

not	输 入			
	0	1	x	z
输 出	1	0	x	x

buf B1 (out1 , out2 , ...,
in) ;



buf 真值表

buf	输 入			
	0	1	x	z
输 出	0	1	x	x

3、三态

有一个输出、一个数据输入和一个输入控制。
如果输入控制信号无效，则三态门的输出为高阻态 Z。

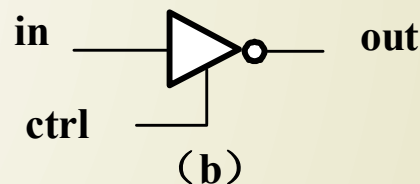
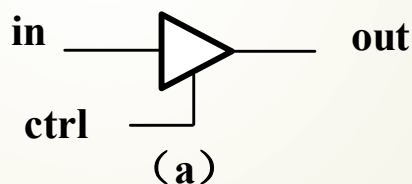


图 4.6.3 三态门元件模型

(a) bufif1

(b) notif1

bufif1 真值

表

bufif1		控制输入			
		0	1	x	z
数据输入	0	z	0	0/z	0/z
	1	z	1	1/z	1/z
	x	z	x	x	x
	z	z	x	x	x

notif1 真值表

notif1		控制输入			
		0	1	x	z
数据输入	0	z	1	1/z	1/z
	1	z	0	0/z	0/z
	x	z	x	x	x
	z	z	x	x	x